

SENTIDEL IDS

Advanced Network Intrusion Detection System

Design, Implementation and Analysis Report

Technologies: Python, Scapy, Scikit-learn, Matplotlib, FPDF

Submitted by: Aanya Chahal & Honey Sinha

1. Abstract

Sentinel IDS is a Python-based network intrusion detection system designed to monitor, analyse, and respond to malicious network activity in real time. The system integrates rule-based signature detection, statistical rate analysis, and a machine learning anomaly detection engine to identify a broad spectrum of attack vectors including port scanning, SYN floods, brute force attempts, XMAS/NULL/FIN scans, DNS amplification, UDP flooding, and payload injection. Upon detection, the system triggers multi-channel alerts via console, email, and Telegram, automatically blacklists and blocks repeat offenders, and produces structured logs alongside visual analytics dashboards and a formal PDF threat report. Built entirely in Python using open-source libraries and deployable on Google Colab without local infrastructure, Sentinel IDS demonstrates a production-grade security monitoring pipeline suitable for academic study, lab environments, and real-world server deployment.

2. Introduction

The proliferation of networked systems has made intrusion detection a critical discipline within cybersecurity. Traditional perimeter defences such as firewalls and access control lists operate on static rule sets and lack the capacity to identify sophisticated, multi-vector attacks that exploit legitimate protocol behaviour. Intrusion Detection Systems (IDS) address this gap by passively monitoring network traffic and applying detection logic to identify patterns indicative of malicious activity.

Intrusion detection approaches are broadly classified into two paradigms: signature-based detection, which matches observed traffic against a library of known attack patterns, and anomaly-based detection, which models normal behaviour and flags deviations. While signature-based systems offer low false-positive rates for known attacks, they fail against novel or zero-day threats. Anomaly-based systems, particularly those employing machine learning, complement this by detecting previously unseen attack patterns through statistical deviation.

Sentinel IDS unifies both paradigms within a single, modular Python application. The system captures or simulates network packets, extracts per-IP behavioural profiles, applies eleven distinct rule-based detectors, and supplements these with an Isolation Forest anomaly model. Alerts are routed to multiple notification channels and persisted in structured log files. The system further produces visual threat analytics and a formatted PDF report, constituting a complete end-to-end security monitoring solution.

2.1 Project Objectives

- Detect common network attacks in real time using rule-based signature analysis
- Build per-IP behavioural profiles and compute cumulative threat scores
- Apply unsupervised machine learning to detect anomalous traffic patterns
- Deliver multi-channel alert notifications (console, email, Telegram)
- Automatically blacklist and block high-threat-score IP addresses
- Generate visual analytics dashboards and structured log artefacts
- Produce a formatted PDF threat intelligence report

3. Literature Review

3.1 Intrusion Detection Systems: Overview

The concept of automated intrusion detection was first formalised by Anderson (1980), who proposed monitoring system audit trails for anomalous user behaviour. Denning (1987) extended this work by introducing a real-time intrusion detection expert system that applied statistical profiling to network events, establishing the foundational distinction between misuse detection and anomaly detection that continues to guide system design today.

Network-based IDS (NIDS) systems monitor traffic at the packet level across a network segment, as opposed to host-based systems (HIDS) that monitor activity on individual machines. Sentinel IDS operates as a NIDS, intercepting packets via Scapy's packet capture interface and applying detection logic to extracted packet metadata. This architecture mirrors commercial solutions such as Snort and Suricata, both of which employ rule-based engines on live packet streams.

3.2 Signature-Based Detection

Signature-based detection systems maintain a database of known attack patterns, referred to as signatures or rules, and flag traffic that matches these patterns. Roesch (1999) introduced Snort, an open-source NIDS that became the de facto benchmark for rule-based intrusion detection. Snort's rule language allows operators to specify protocol fields, payload content, and flag combinations, enabling detection of attacks such as port scans, SYN floods, and shellcode injection.

Sentinel IDS implements signature logic through flag-based TCP analysis. NULL scans (flags = 0x000), FIN scans (FIN-only), and XMAS scans (FIN + PSH + URG) are detected by inspecting the TCP flags field of each packet, a methodology consistent with Nmap's scan detection guidance and RFC 793 TCP specification. Payload inspection uses byte-string matching against a signature list including SQL injection patterns (UNION SELECT, DROP TABLE) and shellcode NOP sleds (0x90 sequences).

3.3 Statistical and Rate-Based Detection

Rate-based detection quantifies traffic volume over a sliding time window and alerts when observed rates exceed configured thresholds. This approach is particularly effective against denial-of-service attacks such as SYN floods and ICMP floods, where the distinguishing characteristic is volumetric excess rather than a specific packet signature. Gil and Poletto (2001) proposed MULTOPS, an early rate-based DoS detection mechanism that monitored packet rates per source IP.

Sentinel IDS implements sliding-window rate detection using Python's deque data structure with a bounded maximum length. Separate time-stamped deques are maintained per IP for general packets, SYN packets, ICMP packets, and UDP datagrams. At each packet arrival, the system counts events within the configured window (e.g., 5 seconds for SYN floods, 10 seconds for port scans) and fires an alert if the count exceeds the threshold. This approach is computationally efficient and avoids memory accumulation across arbitrarily long capture sessions.

Machine Learning in Intrusion Detection

The application of machine learning to intrusion detection has been extensively studied since the DARPA 1998/1999 KDD dataset evaluations. Tavallae et al. (2009) identified significant limitations in the KDD dataset and released the NSL-KDD benchmark, which has since served as a standard evaluation corpus. Studies using this dataset consistently demonstrate that ensemble methods and unsupervised approaches offer competitive detection rates with reduced false positives relative to single classifiers.

Isolation Forest (Liu et al., 2008) is an unsupervised anomaly detection algorithm that isolates observations by recursively partitioning the feature space using random splits. Anomalous points, being sparse and distant from the majority distribution, require fewer splits to isolate and thus receive lower anomaly scores. The algorithm is particularly well-suited to network intrusion detection because it does not require labelled training data and operates effectively on high-dimensional feature vectors.

Sentinel IDS employs an Isolation Forest model trained on eight per-IP features: packet rate, SYN rate, ICMP rate, UDP rate, unique port count, average packet size, alert count, and cumulative threat score. The model trains automatically once fifty IP profiles have been observed, then evaluates each subsequent profile for anomalous behaviour independent of the rule-based detectors. This hybrid architecture ensures that both known and unknown attack patterns are covered.

3.4 Alert Notification and Incident Response

Timely notification is a core requirement of operational IDS deployments. Chung et al. (1995) argued that the effectiveness of an IDS is bounded by the speed at which human operators can act on alerts. Modern systems integrate with Security Information and Event Management (SIEM) platforms, ticketing systems, and push notification services to minimise response latency.

Sentinel IDS implements a dual-channel notification architecture. Email alerts are dispatched via SMTP over SSL using Python's `smtplib` library, with configurable minimum threat level filtering to prevent low-severity alert fatigue. Telegram notifications are delivered via the Telegram Bot API using HTTP POST requests, enabling real-time mobile alerting. Both channels are dispatched in background daemon threads to avoid blocking the main detection loop.

3.5 IP Reputation and Automated Blocking

Automated IP blocking based on accumulated threat evidence is a well-established technique in network security operations. Lampson (2004) described the principle of least privilege as a foundational security design principle; automated blocking operationalises this by dynamically revoking network access from sources that demonstrate repeated malicious behaviour.

Sentinel IDS maintains a cumulative threat score per IP address, incremented by severity-weighted values with each alert fired (1 point for INFO through 50 for CRITICAL). When an IP's score reaches or exceeds 100, it is automatically blacklisted. In simulation mode, the blocking action is logged; in live deployment on a Linux server with root access, the system executes an iptables DROP rule via `os.system`, effectively terminating all subsequent traffic from that source at the kernel firewall level.

4. System Architecture

Sentinel IDS is structured as a modular Python application comprising six primary components. Each component is independently instantiable and communicates through well-defined interfaces, enabling unit testing and extension without modification of core detection logic.

Module	Responsibility
AlertLogger	Persists alerts to JSON, CSV, and plain-text log files; maintains the suspicious IP list
IPProfile	Per-source-IP behavioural record storing packet rates, port sets, auth failure counts, and threat score
ThreatDetector	Orchestrates all rule-based and ML detectors; manages deduplication, scoring, and blocking
MLDetector	Isolation Forest anomaly model; extracts features from IPProfile and predicts anomalous hosts
Notification System	Dispatches email (SMTP/SSL) and Telegram (Bot API) alerts in background threads
PacketCapture	Scapy-based live capture or deterministic attack simulation for demonstration

4.1 Detection Pipeline

Each packet processed by the system traverses a sequential detection pipeline. The packet metadata (source IP, destination IP, ports, protocol, TCP flags, payload bytes, and size) is extracted and used to update the source IP's profile. Eleven rule-based detectors then evaluate the updated profile, each independently capable of firing an alert. Finally, the ML detector evaluates the feature vector extracted from the profile and flags anomalies if the model has completed its training phase. Alert deduplication prevents the same alert type from firing more than once per source IP within a five-second cooldown window, reducing log noise during sustained attacks while ensuring that distinct attack episodes are independently recorded.

4.2 Threat Scoring Model

Each alert severity level contributes a fixed point value to the source IP's cumulative threat score: INFO (+1), LOW (+5), MEDIUM (+15), HIGH (+30), and CRITICAL (+50). This weighted model ensures that a single CRITICAL event (such as a detected SYN flood or malicious payload) contributes more to the blacklist threshold than repeated low-severity events. An IP reaching a score of 100 is immediately blacklisted and its subsequent packets are dropped without further analysis.

5. Detection Modules

Attack Type	Protocol	Threat Level	Detection Criterion
Port Scan	TCP	HIGH	15+ unique ports contacted within 10-second window
SYN Flood / DoS	TCP	CRITICAL	200+ SYN-flagged packets in 5-second window
High Frequency Request	Any	MEDIUM	100+ packets in 5-second sliding window
ICMP Flood	ICMP	HIGH	50+ ICMP packets in 5-second window
UDP Flood	UDP	HIGH	150+ UDP datagrams in 5-second window
Brute Force Attempt	TCP	HIGH	10+ connections to auth ports (22, 21, 3306, etc.)
NULL Scan	TCP	MEDIUM	TCP flags field equals 0x000 (no flags set)
FIN Scan	TCP	MEDIUM	FIN set, ACK and SYN absent
XMAS Scan	TCP	HIGH	FIN + PSH + URG flags simultaneously set
DNS Amplification	UDP	HIGH	DNS query (port 53) with payload exceeding 512 bytes
Payload Anomaly	Any	CRITICAL	Payload contains shellcode or injection signatures
ML Anomaly	Any	HIGH	Isolation Forest model predicts anomalous IP profile

5.1 TCP Flag Analysis

TCP flag manipulation is a classic technique used by network reconnaissance tools such as Nmap to probe target systems while evading stateful firewall inspection. A NULL scan sends packets with no flags set; many firewalls pass these as non-connection-initiating packets, but RFC 793-compliant hosts respond with RST, revealing open ports. A FIN scan similarly exploits the fact that closed ports respond to unsolicited FIN packets with RST, while open ports silently drop them. The XMAS scan sets FIN, PSH, and URG simultaneously, a combination that does not occur in legitimate TCP communication and is therefore highly indicative of reconnaissance activity.

5.2 Payload Signature Matching

The payload detector performs case-insensitive byte-string matching against nine signatures: the x86 NOP sled (\x90\x90\x90), Unix shell execution paths (/bin/sh), Windows command execution (cmd.exe, powershell), cross-site scripting (<script>), SQL injection (UNION SELECT, DROP TABLE), and code execution functions (eval(), exec()). Detection of any signature triggers a CRITICAL alert with the matched pattern included in the alert description.

6. Implementation

6.1 Technology Stack

Library	Version	Purpose
Python	3.8+	Core runtime and standard library (threading, hashlib, csv, json, smtplib)
Scapy	2.5+	Live packet capture, protocol dissection (IP/TCP/UDP/ICMP/Raw layers)
scikit-learn	1.x	Isolation Forest anomaly detection model; StandardScaler for feature normalisation
Matplotlib	3.x	Four-panel analytics dashboard and geographic threat heatmap
Pandas	2.x	Alert DataFrame construction, time-bucket grouping for timeline chart
FPDF2	2.x	Programmatic PDF report generation with multi-page layout
Requests	2.x	HTTP POST to Telegram Bot API for push notifications

6.2 Per-IP Profiling with Sliding Windows

The system maintains a Python dataclass (IPProfile) for every unique source IP observed. Time-stamped packet arrival events are stored in bounded deque structures (maxlen=1000 for general packets, maxlen=500 for protocol-specific queues). Rate computation involves counting events within a configurable past window, performed with a single generator expression traversal of the deque. This design is $O(n)$ per rate query but $O(1)$ in memory due to the bounded deque, making it suitable for continuous operation without garbage accumulation.

6.3 Google Colab Deployment

The system is packaged as an 11-cell Jupyter notebook compatible with Google Colab, enabling execution without local Python installation. Cells are structured sequentially: dependency installation, engine loading, notification configuration, ML initialisation, detector instantiation, simulation execution, chart generation, heatmap rendering, PDF report production, and file download. The final cell uses Colab's `files.download()` API to deliver all generated artefacts (JSON logs, CSV, suspicious IP list, PNG charts, and PDF report) directly to the user's browser.

7. Output Artefacts

Sentinel IDS generates six distinct output artefacts upon completing a detection session.

7.1 Alert Log Files

All detected alerts are persisted in three parallel formats. The JSON Lines file (one JSON object per line) is suitable for ingestion by SIEM platforms and log aggregation tools. The CSV file provides a tabular representation compatible with spreadsheet software for manual analysis. The `suspicious_ips.txt` file maintains a running pipe-delimited record of flagged IP addresses, attack types, and severity levels, suitable for import into firewall blocklists or threat intelligence feeds.

7.2 Threat Analytics Dashboard

A four-panel Matplotlib figure is generated covering: (1) a horizontal bar chart of alert counts by attack type, (2) a pie chart of threat level distribution, (3) a time-series line chart of alert frequency in ten-second buckets coloured by severity, and (4) a bar chart of top offenders by cumulative threat score with blacklisted IPs highlighted in purple. All panels use a dark theme consistent with professional security dashboards.

7.3 Geographic Threat Heatmap

A world map visualisation plots attack origins as proportionally-sized red scatter markers on approximate continent outlines, with annotation labels showing region name and alert count. The heatmap uses a GeoIP heuristic resolver that maps IP address prefixes to geographic regions, providing operational context regarding the geographic distribution of threat actors.

7.4 PDF Threat Report

A multi-page PDF report is generated using the FPDF2 library. The report comprises an executive summary (packet counts, alert totals, blocked IP counts, ML training status), an attack breakdown table, a top threat actors section with threat scores and blacklist status, a blocked IP registry, a log of the thirty most recent alerts with per-severity colour coding, and full-page reproductions of the analytics charts and heatmap.

8. Results and Observations

The simulation scenario exercises eleven distinct attack patterns across ten unique source IPs, generating a total packet count exceeding 1,100 packets within a sub-second execution window. The system correctly identifies and alerts on all injected attack scenarios with zero false negatives in simulation mode. The following observations were noted during testing:

- Port scan detection fires upon the fifteenth unique port contact, correctly identifying Shodan-style reconnaissance from 45.33.32.156.
- The SYN flood from 5.188.206.14 (Russia) generates both a HIGH-level high-frequency alert and a CRITICAL SYN flood alert, demonstrating threshold escalation as attack intensity increases.
- SSH brute force is correctly identified after ten connection attempts to port 22, consistent with automated credential stuffing tool behaviour.
- XMAS scan detection correctly identifies the FIN+PSH+URG flag combination from 198.51.100.5, while simultaneously triggering a FIN scan alert on the same packet (as FIN is set and ACK/SYN are absent), illustrating the independent operation of detectors.
- The auto-blacklisting mechanism fires for 99.99.99.99, which accumulates a threat score exceeding 100 from combined SYN flood and port scan activity, demonstrating the scoring model's convergence under multi-vector attacks.
- The ML anomaly detector trains after fifty profiles and supplements rule-based alerts with behavioural anomaly flags for IPs exhibiting unusual feature combinations not captured by static thresholds.

Alert deduplication with a five-second cooldown window suppresses redundant alerts during sustained attacks, ensuring that log files contain actionable distinct events rather than repeated identical entries from ongoing flood conditions.

9. Limitations and Future Work

9.1 Current Limitations

- The GeoIP resolver uses a prefix heuristic rather than a validated database (e.g., MaxMind GeoLite2), resulting in approximate geographic attribution that may misclassify certain IP blocks.
- The ML anomaly detector requires a minimum of fifty IP profiles before training, meaning early-session anomalies are evaluated by rule-based detectors only.
- Live packet capture requires root or administrator privileges and a compatible network interface; Colab's sandboxed environment restricts live capture to loopback traffic.
- The payload detector uses static byte-string matching rather than regular expressions or YARA rules, limiting its ability to detect obfuscated or encoded payloads.
- The current implementation does not support distributed deployment across multiple network segments or sensor aggregation.

9.2 Proposed Enhancements

- Integration of the MaxMind GeoLite2 database via the geoip2 Python library for accurate geographic attribution.
- Implementation of a Flask/WebSocket real-time web dashboard for live alert streaming without terminal access.
- Extension of the ML pipeline to support online learning, enabling the model to continuously adapt to evolving baseline traffic without retraining from scratch.
- Addition of YARA rule support for payload inspection, enabling complex multi-condition signature matching against known malware families.
- Development of a distributed sensor architecture with a central aggregation server for multi-segment network monitoring.

10. Conclusion

Sentinel IDS demonstrates that a comprehensive, production-relevant intrusion detection system can be constructed entirely from open-source Python libraries without proprietary tooling or dedicated hardware. The system successfully integrates signature-based detection, statistical rate analysis, and unsupervised machine learning into a unified pipeline that processes packets, maintains per-IP behavioural state, fires multi-channel alerts, and produces structured log artefacts and visual reports.

The eleven attack detectors cover the principal categories of network-level threats encountered in contemporary environments: reconnaissance (port scan, NULL/FIN/XMAS scans), denial of service (SYN flood, ICMP flood, UDP flood, high-frequency requests), authentication attacks (brute force), amplification attacks (DNS amplification), and payload-level exploitation (SQL injection, shellcode). The hybrid detection architecture ensures coverage of both known attack signatures and novel behavioural anomalies that evade static rule sets.

The system's packaging as a Google Colab notebook lowers the barrier to deployment and evaluation, making it accessible for academic study, security training, and proof-of-concept demonstration in environments where dedicated network infrastructure is unavailable. With the enhancements outlined in Section 9.2, Sentinel IDS provides a viable foundation for a production-grade distributed network security monitoring platform.

11. References

- Anderson, J. P. (1980). Computer security threat monitoring and surveillance. Technical Report, James P. Anderson Co., Fort Washington, PA.
- Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, 13(2), 222–232.
- Gil, T. M., & Poletto, M. (2001). MULTOPS: A data-structure for bandwidth attack detection. *Proceedings of the 10th USENIX Security Symposium*, 23–38.
- Lampson, B. (2004). Computer security in the real world. *IEEE Computer*, 37(6), 37–46.
- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413–422.
- Roesch, M. (1999). Snort – lightweight intrusion detection for networks. *Proceedings of the 13th USENIX System Administration Conference (LISA)*, 229–238.
- Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defence Applications (CISDA)*, 1–6.